Solution: Basic Calculator II

Problem Statement

Given a string `s`, representing the `mathematical expression`, `evaluate` the expression and return the `integer` value.

The integer division should truncate toward zero.

**Note:* You are `not allowed` to use the built-in functions like `eval()`.

Examples

Example 1:

- **Input**: `"8*5/4+3-2"`
- **Expected Output**: `11`
- **Justification**: The calculation proceeds as follows: `8*5 = 40`, `40/4 = 10`, `10+3 = 13`, `13-2 = 11`.

Example 2:

- **Input**: `"10+20/5*3"`
- **Expected Output**: `22`
- **Justification**: The order of operations gives `20/5 = 4`, `4*3 = 12`, and `10+12 = 22`.

Example 3:

- **Input**: `"50-25+5*3/2*2-10/2"`
- **Expected Output**: `34`
- **Justification**: Following the order of operations: `50 - 25 = 25`, `(15/2) = 7` (rounded down), `7*2 = 14`, `25+14 = 39`, `10/2 = 5`, and `39 - 5 = 34`.

Solution

To solve this problem, our approach blends linear parsing with immediate evaluation of arithmetic operations. This methodology effectively addresses the challenge of respecting the standard order of operations (multiplication and division prior to addition and subtraction) within a single pass through the expression. The logic behind this approach is to evaluate and accumulate partial results as soon as possible, especially when dealing with multiplication and division, which have higher precedence.

The efficiency of this method lies in its ability to process the expression in a single sweep, thereby minimizing the need for backtracking or re-evaluating portions of the expression. By updating the cumulative result and the last evaluated number at each step, based on the current operation, the algorithm ensures that the most recent operation is always applied correctly. This real-time processing approach not only maintains the correct order of operations but also streamlines the computational process, leading to a solution that is both time and space-efficient.

Step-by-Step Algorithm

1. Initialize `currentNumber`, `lastNumber`, and `result` to 0, and set `operation` to '+'.
2. Iterate over each character in the input string:
   - If the character is a digit, update `currentNumber` accordingly.
   - If the character is a non-digit, non-space, or it's the end of the string:
     - If `operation` is '+' or '-', add `lastNumber` to `result`. Update `lastNumber` to `currentNumber` or `-currentNumber` based on `operation`.
     - If `operation` is '*', multiply `lastNumber` with `currentNumber`.
     - If `operation` is '/', divide `lastNumber` by `currentNumber`.
     - Update `operation` to the current character.
     - Reset `currentNumber` to 0.
3. After the loop, add `lastNumber` to `result` for the final total.

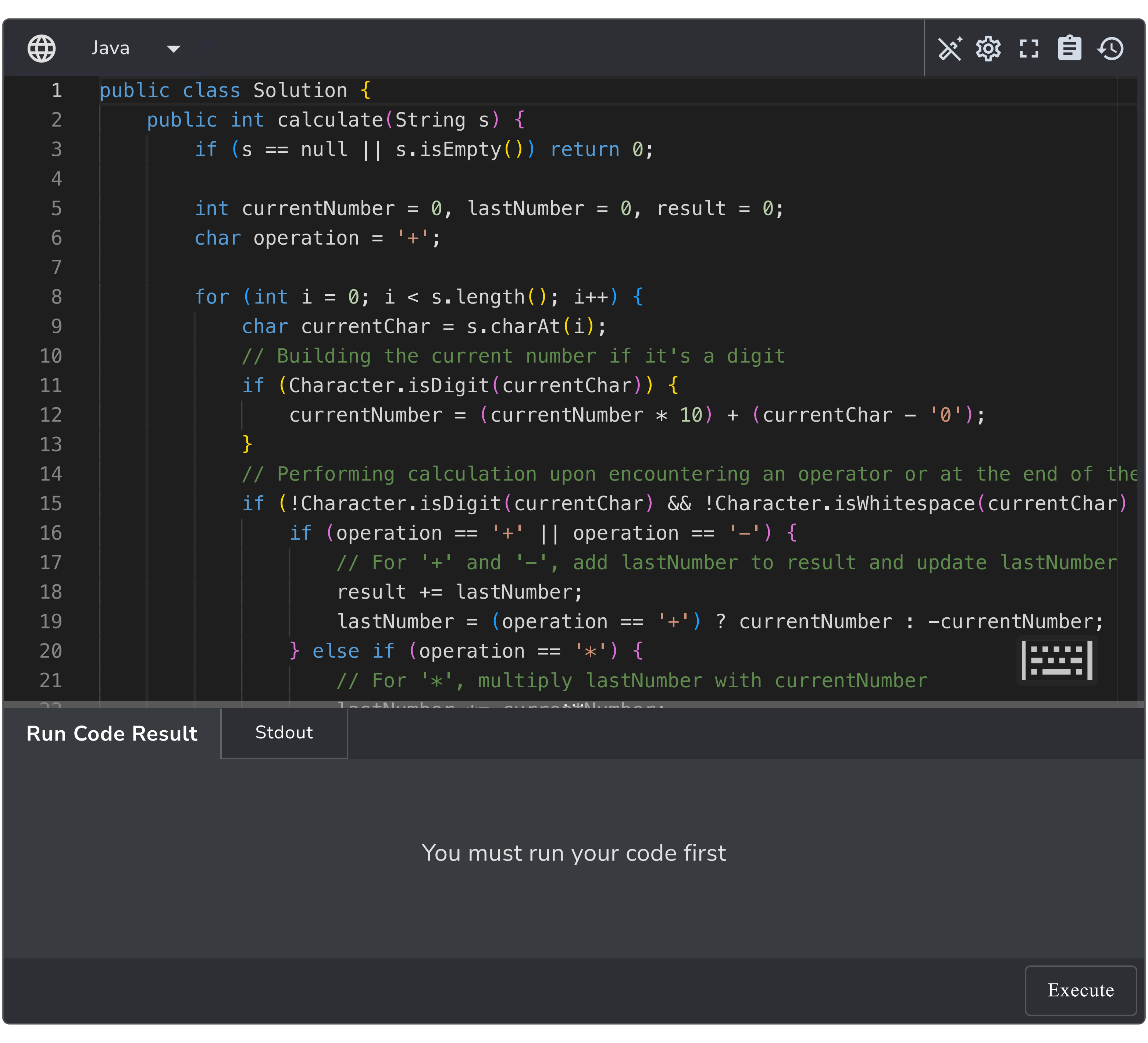
Algorithm Walkthrough

let's consider input: `"50-25+5*3/2*2-10/2"`

Certainly, let's correct the walkthrough of the code with the input `"50-25+5*3/2*2-10/2"`. We'll carefully trace through the arithmetic and ensure accurate computation at each step.

1. **Initialization:**
   - `currentNumber = 0`, `lastNumber = 0`, `result = 0`, `operation = '+'`.
2. **Processing each character**:
   - Iterate through each character of the string.
3. **First Character '5'**:
   - `currentNumber` becomes 5.
4. **Second Character '0'**:
   - `currentNumber` becomes 50.
5. **Third Character '-'**:
   - Operation is `+`, so `lastNumber = 50` (as `currentNumber` is 50).
   - `result` becomes 0 (0 + 0).
   - `operation` updated to '-'.
   - `currentNumber` reset to 0.
6. **Next Characters '25'**:
   - `currentNumber` becomes 25.
7. **Character '+'**:
   - Operation is '-', so `lastNumber = -25`.
   - `result` becomes 50 (0 + 50).
   - `operation` updated to '+'.
   - `currentNumber` reset to 0.
8. **Next Characters '5'**:
   - `currentNumber` becomes 5.
9. **Character '*'**:
   - Operation is '+', `result` becomes 25 (50 - 25).
   - Update `lastNumber = 5
   - `operation` updated to '*'.
   - `currentNumber` reset to 0.
10. **Next Characters '3'**:
    - `currentNumber` becomes 3.
11. **Character '/'**:
    - Operation is '*', so `lastNumber` becomes 15 (5 * 3).
    - `operation` updated to '/'.
    - `currentNumber` reset to 0.
12. **Next Characters '2'**:
    - `currentNumber` becomes 2.
13. **Character '*'**:
    - Operation is '/', so `lastNumber` becomes 7 (15 / 2, rounded down for integer division).
    - `operation` updated to '*'.
    - `currentNumber` reset to 0.
14. **Next Characters '2'**:
    - `currentNumber` becomes 2.
15. **Character '-'**:
    - Operation is '*', so `lastNumber` becomes 14 (7 * 2).
    - `result` becomes 39 (25 + 14).
    - `operation` updated to '-'.
    - `currentNumber` reset to 0.
16. **Next Characters '10'**:
    - `currentNumber` becomes 10.
17. **Character '/'**:
    - Operation is '-', so `lastNumber` becomes -10.
    - `result` remains 39.
    - `operation` updated to '/'.
    - `currentNumber` reset to 0.
18. **Next Characters '2'**:
    - `currentNumber` becomes 2.
19. **End of String**:
    - Operation is '/', so `lastNumber` becomes -5 (10 / 2, integer division).
    - `result` becomes 34 (39 - 5).
20. **Final Calculation**:
    - `result` is 34.

Code



```java
public class Solution {
    public int calculate(String s) {
        if (s == null || s.isEmpty()) return 0;

        int currentNumber = 0, lastNumber = 0, result = 0;
        char operation = '+';

        for (int i = 0; i < s.length(); i++) {
            char currentChar = s.charAt(i);
            // Building the current number if it's a digit
            if (Character.isDigit(currentChar)) {
                currentNumber = (currentNumber * 10) + (currentChar - '0');
            }
            // Performing calculation upon encountering an operator or at the end of the
            if (!Character.isDigit(currentChar) && !Character.isWhitespace(currentChar)
                if (operation == '+' || operation == '-') {
                    // For '+' and '-', add lastNumber to result and update lastNumber
                    result += lastNumber;
                    lastNumber = (operation == '+') ? currentNumber : -currentNumber;
                } else if (operation == '*') {
                    // For '*', multiply lastNumber with currentNumber
```

You must run your code first

Complexity Analysis

- **Time Complexity:** The algorithm has a linear time complexity of `O(N)`, where N represents the length of the input string. This is because the algorithm processes each character in the string exactly once.
- **Space Complexity:** The space complexity of the algorithm is `O(1)`, or constant space complexity. This is due to the use of a fixed number of variables regardless of the size of the input string.

← Previous
Basic Calculator II (Medium)

Next →
Nested List Weight Sum (Medium)

☐ Mark as Completed